

ASSIGNMENT 6

QUESTION 1

Generate a dataset with atleast seven highly correlated columns and a target variable. Implement Ridge Regression using Gradient Descent Optimization. Take different values of learning rate (such as 0.0001,0.001,0.01,0.1,1,10) and regularization parameter (10-15,10-10,10-5,10-3,0,1,10,20). Choose the best parameters for which ridge regression cost function is minimum and R2_score is maximum.

SOLUTION

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("./data.csv")
print(df.head())

def ridge_regression_gradient_descent(X, y, learning_rate, reg_param, num_iterations):
    m, n = X.shape
    theta = np.zeros(n)
    cost_history = []

    for _ in range(num_iterations):
        predictions = X.dot(theta)

        errors = predictions - y

        theta_for_reg = theta.copy()
        theta_for_reg[0] = 0

        gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg

        theta -= learning_rate * gradient

        cost = (1 / (2 * m)) * np.sum(errors**2) + (reg_param / (2 * m)) * np.sum(
            theta_for_reg**2
        )
        cost_history.append(cost)

    return theta, cost_history

X = df.drop("Target_Variable", axis=1).values
y = df["Target_Variable"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```

X_train = np.c_[np.ones(X_train.shape[0]), X_train]
X_test = np.c_[np.ones(X_test.shape[0]), X_test]

learning_rates = [0.0001, 0.001, 0.01, 0.1, 1, 10]
reg_params = [1e-15, 1e-10, 1e-5, 1e-3, 0, 1, 10, 20]

best_r2 = -np.inf
best_params = None
best_theta = None
lowest_cost = np.inf

for lr in learning_rates:
    for reg in reg_params:
        theta, cost_history = ridge_regression_gradient_descent(
            X_train, y_train, lr, reg, 1000
        )

        if np.isnan(theta).any():
            continue

        y_pred = X_test.dot(theta)
        r2 = r2_score(y_test, y_pred)
        final_cost = cost_history[-1]

        if r2 > best_r2:
            best_r2 = r2
            best_params = (lr, reg)
            best_theta = theta
            lowest_cost = final_cost

print(f"Best Learning Rate: {best_params[0]}")
print(f"Best Regularization Parameter: {best_params[1]}")
print(f"Best R2 Score: {best_r2:.4f}")
print(f"Cost at Minimum: {lowest_cost:.4f}")
print(f"Best Coefficients (Thetas): \n{best_theta}")

```

OUTPUT

	Feature_1	Feature_2	Feature_3	Feature_4	Feature_5	Feature_6	\
0	10.5	21.2	110.4	5.2	52.1	9.5	
1	12.1	24.5	126.3	6.0	60.8	11.2	
2	9.8	19.7	103.2	4.9	49.2	8.9	
3	15.3	30.8	158.1	7.6	76.9	14.1	
4	11.2	22.6	117.5	5.5	56.3	10.4	

	Feature_7	Target_Variable
0	30.8	150.2
1	35.9	172.5
2	29.1	140.8
3	45.5	218.9
4	33.2	160.4

```

/tmp/ipykernel_4018/2238133972.py:28: RuntimeWarning: overflow encountered in square
  cost = (1 / (2 * m)) * np.sum(errors**2) + (reg_param / (2 * m)) * np.sum(
/home/himanshu/assignments/.venv/lib/python3.11/site-packages/numpy/_core/fromnumeric.py:86:
RuntimeWarning: overflow encountered in reduce
  return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
/tmp/ipykernel_4018/2238133972.py:29: RuntimeWarning: overflow encountered in square
  theta_for_reg**2

```

```

/tmp/ipykernel_4018/2238133972.py:24: RuntimeWarning: overflow encountered in dot
  gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg
/tmp/ipykernel_4018/2238133972.py:17: RuntimeWarning: invalid value encountered in dot
  predictions = X.dot(theta)
/tmp/ipykernel_4018/2238133972.py:24: RuntimeWarning: invalid value encountered in dot
  gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg
/tmp/ipykernel_4018/2238133972.py:28: RuntimeWarning: invalid value encountered in scalar
multiply
  cost = (1 / (2 * m)) * np.sum(errors**2) + (reg_param / (2 * m)) * np.sum(
/tmp/ipykernel_4018/2238133972.py:24: RuntimeWarning: invalid value encountered in multiply
  gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg
/tmp/ipykernel_4018/2238133972.py:17: RuntimeWarning: overflow encountered in dot
  predictions = X.dot(theta)
Best Learning Rate: 0.1
Best Regularization Parameter: 1e-15
Best R2 Score: 0.9999
Cost at Minimum: 0.1280
Best Coefficients (Thetas):
[173.7          4.57991815   4.55307975   4.45287262   4.75663862
  4.68334736   3.97350161   4.85169357]

```

QUESTION 2

Load the Hitters dataset from the following link

https://drive.google.com/file/d/1qzCKF6JKKMB0p7uL_ILy8tdmRk3vE_bG/view?usp=sharing

- Pre-process the data (null values, noise, categorical to numerical encoding)
- Separate input and output features and perform scaling
- Fit a Linear, Ridge (use regularization parameter as 0.5748), and LASSO (use regularization parameter as 0.5748) regression function on the dataset.
- Evaluate the performance of each trained model on test set. Which model performs the best and Why?

SOLUTION

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error

df = pd.read_csv("hitters.csv")

df = df.dropna(subset=["Salary"])
df = pd.get_dummies(df, columns=["League", "Division", "NewLeague"], drop_first=True)

X = df.drop("Salary", axis=1).values
y = df["Salary"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=69
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

ridge_reg = Ridge(alpha=0.5748)
ridge_reg.fit(X_train_scaled, y_train)

lasso_reg = Lasso(alpha=0.5748, max_iter=10000)
lasso_reg.fit(X_train_scaled, y_train)

models = {
    "Linear Regression": lin_reg,
    "Ridge Regression": ridge_reg,
    "Lasso Regression": lasso_reg,
}

print(f"{'Model':<20} | {'R2 Score':<10} | {'RMSE':<10}")

best_model_name = ""
best_r2 = -np.inf

for name, model in models.items():
    y_pred = model.predict(X_test_scaled)

    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    print(f"{'name':<20} | {'r2':.4f} | {'rmse':.2f}")

    if r2 > best_r2:
        best_r2 = r2
        best_model_name = name

print(f"The best performing model is: {best_model_name}")

```

OUTPUT

Model	R2 Score	RMSE
Linear Regression	0.3226	338.08
Ridge Regression	0.3226	338.10
Lasso Regression	0.3212	338.44

The best performing model is: Linear Regression

QUESTION 3

Explore Ridge Cross Validation (RidgeCV) and Lasso Cross Validation (LassoCV)

SOLUTION

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV

# load boston is deprecated :(
# df = load_boston(as_frame=True).frame
data_url = "https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv"
df = pd.read_csv(data_url)

```

```

X = df.drop("medv", axis=1)
y = df["medv"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

alphas_to_test = [0.01, 0.1, 1.0, 10.0, 20.0, 50.0, 100.0]

ridge_cv = RidgeCV(alphas=alphas_to_test, cv=5)

ridge_cv.fit(X_train_scaled, y_train)

print(f"Best Alpha found by RidgeCV: {ridge_cv.alpha_}")
print(f"RidgeCV Train Score (R2): {ridge_cv.score(X_train_scaled, y_train):.4f}")
print(f"RidgeCV Test Score (R2): {ridge_cv.score(X_test_scaled, y_test):.4f}")

lasso_cv = LassoCV(n_alphas=100, cv=5, random_state=42)

lasso_cv.fit(X_train_scaled, y_train)

print(f"Best Alpha found by LassoCV: {lasso_cv.alpha_}")
print(f"LassoCV Train Score (R2): {lasso_cv.score(X_train_scaled, y_train):.4f}")
print(f"LassoCV Test Score (R2): {lasso_cv.score(X_test_scaled, y_test):.4f}")

coefs = pd.Series(lasso_cv.coef_, index=X.columns)
print(f"Number of features ignored by Lasso (Coef = 0): {sum(coefs == 0)}")
print("Features with 0 weight:", list(coefs[coefs == 0].index))

```

OUTPUT

```

Best Alpha found by RidgeCV: 1.0
RidgeCV Train Score (R2): 0.7509
RidgeCV Test Score (R2): 0.6685
Best Alpha found by LassoCV: 0.006863892263379668
LassoCV Train Score (R2): 0.7508
LassoCV Test Score (R2): 0.6684
Number of features ignored by Lasso (Coef = 0): 0
Features with 0 weight: []
/home/himanshu/assignments/.venv/lib/python3.11/site-packages/sklearn/linear_model/_coordinate_descent.py:1622: FutureWarning: 'n_alphas' was deprecated in 1.7 and will be removed in 1.9. 'alphas' now accepts an integer value which removes the need to pass 'n_alphas'. The default value of 'alphas' will change from None to 100 in 1.9. Pass an explicit value to 'alphas' and leave 'n_alphas' to its default value to silence this warning.
  warnings.warn(

```